

HACKER ATTACK

TESTES DE SEGURANÇA



LISTA DE FIGURAS

Figura 1 – Dashboard Horusec SAST	7
Figura 2 – Ilustração do Horusec pela Zup	9
Figura 3 – Instalação do Horusec I.....	11
Figura 4 – Instalação do Horusec II.....	12

EMANIP

LISTA DE QUADROS

Quadro 1 – Diferenças entre SAST e DAST	8
---	---

EMANIP

SUMÁRIO

TESTES DE SEGURANÇA AUTOMATIZADOS	5
1 INTRODUÇÃO	5
2 O QUE SÃO TESTES DE SEGURANÇA ESTÁTICOS	5
3 COMO O SAST FUNCIONA	5
4 E QUAIS PROBLEMAS O SAST RESOLVE?	7
5 TIPOS DE ANÁLISE (SAST, DAST, IAST)	8
6 E ONDE ENCONTRAR UMA FERRAMENTA QUE FAÇA ESSA ANÁLISE?	9
7 LAB INSTALAÇÃO HORUSEC	11
8 CONCLUSÃO	12
REFERÊNCIAS	13

TESTES DE SEGURANÇA AUTOMATIZADOS

1 INTRODUÇÃO

Uma das atividades mais corriqueiras na vida de um profissional que atua com segurança no desenvolvimento de aplicações é a identificação e o tratamento de vulnerabilidades presentes no código-fonte. Esta atividade pode ser desempenhada tanto por um analista de segurança, quanto pelo próprio desenvolvedor, caso este esteja munido dos conhecimentos necessários relacionados à cibersegurança. O importante é que as análises (manuais ou automatizadas) sejam realizadas e que seus resultados forneçam os subsídios necessários para haja correção dessas vulnerabilidades.

2 O QUE SÃO TESTES DE SEGURANÇA ESTÁTICOS

Os testes de segurança estáticos (SAST), do inglês *Static Application Security Testing*, são um conjunto de tecnologias projetadas para analisar o código-fonte ou *bytes code* (binários) para que os aspectos de design que são indicativos de vulnerabilidades de segurança sejam previamente detectados. De um modo geral, esses testes servem para ajudar a descartar a possibilidade de o código possuir alguma vulnerabilidade ou defeito, além de garantir que a aplicação seja executada sem problemas após o seu desenvolvimento.

Muitos incidentes de segurança são causados por invasores que exploram vulnerabilidades de softwares, por essa razão, os desenvolvedores precisam sempre estar atentos aos bugs desde o início do projeto para que os riscos à segurança da que as organizações enfrentam sejam reduzidos.

3 COMO O SAST FUNCIONA

O objetivo aqui não é explicar detalhadamente os mecanismos e processos internos de como estas ferramentas funcionam. O intuito é criar uma base de conhecimento que possa ajudar no entendimento do propósito dessas ferramentas de

análise automatizadas de segurança, isso significa, compreender para que elas servem e em quais situações utilizá-las.

As ferramentas SAST, basicamente, analisam os códigos das aplicações em um modelo *inside out*, ou seja, de dentro para fora, examinando pontos específicos do código, em um estado de não execução. Por essa razão, é importante compreender que nessas análises a probabilidade de existirem casos de falso-positivo e falso-negativo é relativamente alta, tendo em consideração que o contexto pode ser insuficientemente para eliminar essas possibilidades.

Em uma análise SAST - seguem algumas das seguintes regras:

1) Análise de controle de fluxo

A análise de controle de fluxo valida a ordem das operações para buscar por padrões de sequências que possam ser prejudiciais ao código. Podemos citar, como exemplos, falhas de transmissão de cookies, variáveis que não foram iniciadas ou mesmo *race conditions*.

2) Análise estrutural

Com base na linguagem que o desenvolvedor está usando, as ferramentas analisam a estrutura do código, tomando como base as melhores práticas para a linguagem. Essa avaliação leva em consideração técnicas, estilo e práticas de desenvolvimento seguro. Essa análise visa, portanto, identificar pontos fracos na criação das classes, nas declarações e no uso de variáveis.

3) Análise semântica

Essas ferramentas usam como base uma estrutura de análise semântica: uma funcionalidade semelhante ao grep do linux, mas de uma forma bem limitada. As ferramentas de SAST podem analisar o código buscando por contextos específicos, como os métodos de uma classe: `eval()`, `execute()` etc.

4) Análise de configuração

A análise de configuração verifica os arquivos de configuração do aplicativo, como JSON ou arquivos de propriedades. Ela garante que a configuração esteja alinhada às práticas e políticas de segurança definidas pela organização, por exemplo, definindo uma página de erro padrão para o aplicativo Web.

4 E QUAIS PROBLEMAS O SAST RESOLVE?

O SAST, ou teste estático de segurança, funciona bem no começo do ciclo de vida de desenvolvimento de software (SDLC), pois não exige uma aplicação funcional e pode agir sem que o código seja executado. Ele ajuda o desenvolvedor a localizar vulnerabilidades bem no começo do desenvolvimento e solucionar problemas rapidamente sem obstruir as compilações ou passar vulnerabilidades para a versão final do aplicativo.

As ferramentas SAST garantem *feedback* em tempo real aos desenvolvedores, à medida que codificam, ajudando-os a identificar falhas antes de transferirem o código para a próxima fase do ciclo de vida do desenvolvimento do software. Algumas ferramentas apontam a localização exata das vulnerabilidades e destacam o código arriscado, fornecendo orientações detalhadas sobre como reparar problemas e o melhor local no código para corrigi-los.

Com o SAST, também é possível criar relatórios personalizados, que podem ser exportados offline e rastreados por meio de painéis. É importante reforçar que as ferramentas SAST devem ser executadas no aplicativo constantemente, durante compilações diárias ou mensais, toda vez que o código é analisado ou durante o lançamento de um código.

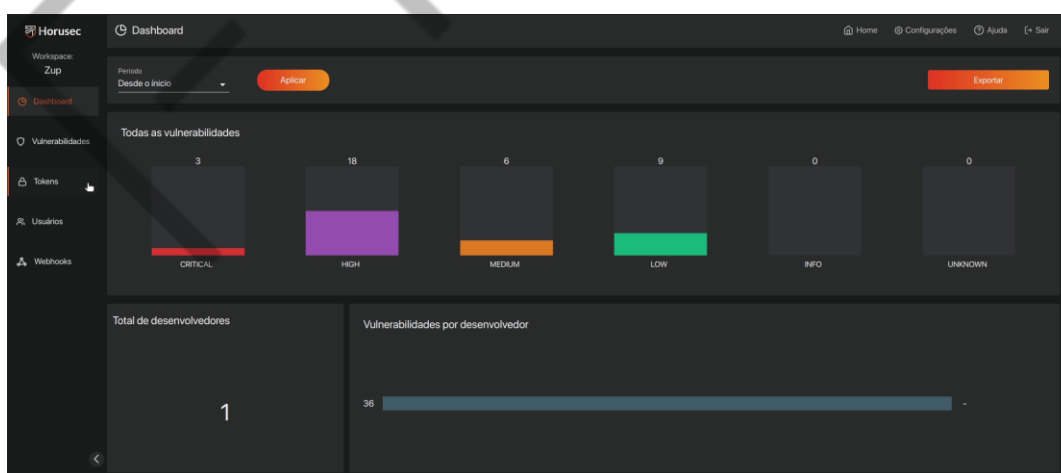


Figura 1 – Dashboard Horusec SAST
Fonte: ZUP. horusec.io (2021)

5 TIPOS DE ANÁLISE (SAST, DAST, IAST)

As siglas até podem ser parecidas, entretanto, há muitas diferenças entre cada uma delas! Vamos lá?

O SAST como já foi apresentado, é um acrônimo que em português significa, teste estático de segurança de aplicação. Ele se refere às ferramentas que analisam o código fonte dos sistemas. Conforme aprendemos, estes testes são realizados antes que o sistema esteja em produção e somente no código fonte.

Já quando falamos de DAST (*Dynamic Application Security Testing*), em português, teste dinâmico de segurança de aplicação, nos referimos às ferramentas que testam interfaces expostas em busca de vulnerabilidades. O DAST é recomendado para encontrar vulnerabilidades visíveis externamente. Para isso, a URL já é suficiente para o teste ser realizado ou o binário ser executado. Um bom exemplo de ferramenta de DAST.

A desvantagem é que o DAST depende de especialistas para programá-lo. Por isso, acaba sendo um pouco difícil de ser escalado nas organizações. Apesar de que, uma vez escrito, pode ser 100% automatizado.

O quadro a seguir traz as diferenças entre as tecnologias:

SAST	DAST
Usado para teste de segurança de caixa branca.	Usado para teste de segurança de caixa preta.
Age analisando o código-fonte, sem ser necessária a execução do código.	Age enquanto o aplicativo está em execução.
As vulnerabilidades são encontradas antes do desenvolvimento e são menos caras para consertar.	Encontra problemas que não podem ser identificados em uma análise estática.
Não consegue identificar problemas relacionados ao tempo e ao ambiente.	As vulnerabilidades são encontradas após o desenvolvimento e são mais caras para consertar.
É capaz de dar suporte a todos os tipos de software.	Consegue identificar problemas relacionados ao tempo e ao ambiente.

Quadro 1 – Diferenças entre SAST e DAST
Fonte: Elaborado pelo autor.

Enquanto isso, o IAST (*Interactive Application Security Restinga*), em português, teste interativo de segurança de aplicação – é basicamente a combinação

dos modelos de testes estáticos e dinâmicos (isto é, SAST e DAST) e, portanto, é a ferramenta que apresenta os melhores resultados.

Uma das principais vantagens do IAST é que existe a opção de ele ser feito em conjunto com um profissional de segurança. Com isso, ele se torna o melhor tipo de teste em questão de taxa de falso positivo, visto que conta com uma interação humana.

6 E ONDE ENCONTRAR UMA FERRAMENTA QUE FAÇA ESSA ANÁLISE?

Para o *Lab* deste capítulo, selecionamos uma ferramenta *open source* desenvolvida pelo time de segurança da Zup, que realiza análise estática de códigos para identificar falhas de segurança durante o processo de desenvolvimento. O Horusec é uma ferramenta *open source* que orquestra outras ferramentas de segurança e identifica falhas de segurança ou vulnerabilidades em projetos, centralizando todos os resultados em um banco de dados para análise e geração de métricas. Atualmente, o Horusec seleciona as linguagens e ferramentas a serem utilizadas no projeto de acordo com a stack disponível.



Figura 2 – Ilustração do Horusec pela Zup
Fonte: Zup Innovation - Horusec (2022)

O Horusec realiza análise SAST seguindo como referência OWASP TOP Ten para encontrar possíveis vulnerabilidades em sua base de código. Em sua última versão, o suporte a linguagens e ferramentas para a análise são:

- Python
- Ruby
- Javascript/Typescript
- GoLang
- C#
- Java
- Kotlin
- Kubernetes
- Terraform
- Leaks
- Leaks(optional search in git history)
- PHP
- C/C++
- HTML
- JSON
- Dart
- Shell Script
- Elixir

A ferramenta também conta com o Leaks, que é busca de vazamentos de chaves e falhas de segurança em todos os arquivos do seu projeto, e ainda no histórico do Git. O Horusec pode ser usado pelo desenvolvedor através da CLI e pela equipe de DevSecOps em esteiras de CI/CD.

Para iniciar a jornada pela ferramenta e ter acesso à sua documentação, acesse a Url oficial: <https://horusec.io/site/>.

7 LAB INSTALAÇÃO HORUSEC

Nesse LAB realizaremos a instalação da versão CLI do Horusec, CLI (sigla para interface de linha de comando) é o programa que processa os comandos de um software ou qualquer programa computacional por meio de texto.

Horusec-CLI é a parte de código que permite rodar os comandos de análise de segurança e de identificação e/ou classificação de vulnerabilidades. Esta instalação é recomendada se você deseja usar o Horusec para fazer análises ou verificar as vulnerabilidades de projeto de aplicação.

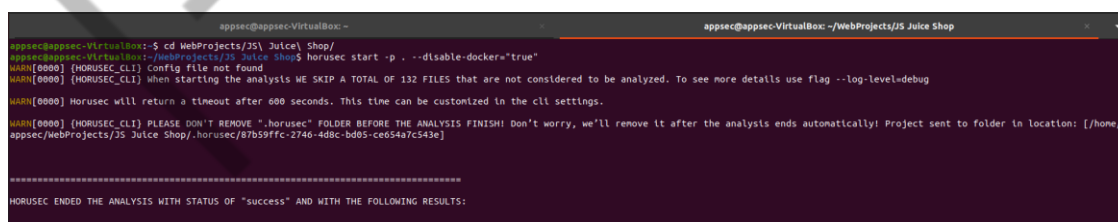
Para a instalação no Linux, os comandos necessários para instalar o Horusec localmente são:

```
curl -fsSL https://raw.githubusercontent.com/ZupIT/horusec/main/deployments/scripts/install.sh | bash -s latest
```

Após a instalação execute o comando abaixo no local onde está seu projeto:

```
horusec start -p . --disable-docker="true"
```

Isso fará com que o scan se inicie sem a dependência de uma imagem Docker.



```
appsec@appsec-VirtualBox: ~  
appsec@appsec-VirtualBox:~$ cd WebProjects/JS/ Juice Shop/  
appsec@appsec-VirtualBox:~/WebProjects/JS/ Juice Shop$ horusec start -p . --disable-docker="true"  
WARN[0000] (HORUSEC_CLI) config file not found  
WARN[0000] (HORUSEC_CLI) when starting the analysis WE SKIP A TOTAL OF 132 FILES that are not considered to be analyzed. To see more details use flag --log-level=debug  
WARN[0000] Horusec will return a timeout after 600 seconds. This time can be customized in the cli settings.  
WARN[0000] (HORUSEC_CLI) PLEASE DON'T REMOVE ".horusec" FOLDER BEFORE THE ANALYSIS FINISH! Don't worry, we'll remove it after the analysis ends automatically! Project sent to folder in location: [/home/  
appsec/WebProjects/JS/ Juice Shop/.horusec/87b59ffc-2740-4d8c-bd05-ce654a7c343e]  
  
*****  
HORUSEC ENDED THE ANALYSIS WITH STATUS OF "success" AND WITH THE FOLLOWING RESULTS:  
*****
```

Figura 3 – Instalação do Horusec I

Fonte: Elaborada pelo autor (2022)

Em poucos segundos, os resultados da análise Sast serão apresentados na tela do terminal, conforme imagem abaixo:

```
SecurityTool: HorusecEngine
Confidence: HIGH
File: /home/appsec/webProjects/JS Juice Shop/server.ts
Code: app.options('*', cors())
RuleID: HS-JAVASCRIPT-19
Details: Having a permissive Cross-Origin Resource Sharing policy
Same origin policy in browsers prevents, by default and for security-reasons, a javascript frontend to perform a cross-origin HTTP request to a resource that has a different origin (domain, protocol, or port) from its own. The requested target can append additional HTTP headers in response, called CORS, that act like directives for the browser and change the access control policy / relax the same origin policy. The Access-Control-Allow-Origin header should be set only for a trusted origin and for specific resources. For more information checkout the OWASP A6:2017 (https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration.html) advisory.
Type: Vulnerability
ReferenceHash: 62f89a63b80e4585847d6468d7e65a8292e57f198a2bf9bef01d83a7f2f9bc2c
=====
Language: JavaScript
Severity: LOW
Line: 145
Column: 18
SecurityTool: HorusecEngine
Confidence: HIGH
File: /home/appsec/webProjects/JS Juice Shop/server.ts
Code: app.use(cors())
RuleID: HS-JAVASCRIPT-19
Details: Having a permissive Cross-Origin Resource Sharing policy
Same origin policy in browsers prevents, by default and for security-reasons, a javascript frontend to perform a cross-origin HTTP request to a resource that has a different origin (domain, protocol, or port) from its own. The requested target can append additional HTTP headers in response, called CORS, that act like directives for the browser and change the access control policy / relax the same origin policy. The Access-Control-Allow-Origin header should be set only for a trusted origin and for specific resources. For more information checkout the OWASP A6:2017 (https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration.html) advisory.
Type: Vulnerability
ReferenceHash: 58a0d0e9cb52e904a2f97ec73201fc9c4a4a3d3898f8d5684f42288d3c5af8d5
=====
In this analysis, a total of 462 possible vulnerabilities were found and we classified them into:
Total of Vulnerability HIGH is: 114
Total of Vulnerability MEDIUM is: 11
Total of Vulnerability LOW is: 4
Total of Vulnerability CRITICAL is: 333
=====
warn[0017] (HORUSEC_CLI) No authorization token was found, your code it is not going to be sent to horusec. Please enter a token with the -a flag to configure and save your analysis
warn[0017] (HORUSEC_CLI) 462 VULNERABILITIES WERE FOUND IN YOUR CODE SENT TO HORUSEC, TO SEE MORE DETAILS USE THE LOG LEVEL AS DEBUG AND TRY AGAIN
warn[0017] (HORUSEC_CLI) Horusec not show info vulnerabilities in this analysis, to see info vulnerabilities add option "--information-severity=true". For more details use (horusec start --help) command
appsec@appsec-VirtualBox: /webProjects/JS Juice Shop$
```

Figura 4 – Instalação do Horusec II

Fonte: Elaborada pelo autor (2022)

Conforme o resultado da análise, o projeto Juice Shop da owasp, disponível em <https://owasp.org/www-project-juice-shop/> foram detectadas um total de 462 vulnerabilidades, os detalhes de cada uma podem ser analisados ao subir as linhas do terminal.

8 CONCLUSÃO

Os testes automatizados são poderosíssimos e talvez por isso essas ferramentas sejam consideradas por muitos como essenciais para o desenvolvimento seguro de software. Contudo, a ideia aqui foi apresentá-las para que façam parte do agregado de ferramentas que um profissional de cibersegurança deve compreender.

REFERÊNCIAS

CONTACTA SEGURANÇA DIGITAL. **SAST, DAST e IAST: entenda a diferença entre esses testes de vulnerabilidade de aplicativos.** 2022. Disponível em: <<https://www.contacta.com.br/sast-dast-e-iaast-entenda-a-diferenca-entre-esses-testes-de-vulnerabilidade-de-aplicativos/>>. Acesso em: 27 fev 2022.

CONVISO. **Diferença entre Sast e Dast.** 2022. Disponível em: <<https://blog.convisoappsec.com/diferenca-entre-code-review-e-sast/>>. Acesso em: 27 fev 2022.

ZUP. I.T Innovation. **Desenvolvimento Seguro com Horusec.** 2022. Disponível em: <<https://www.zup.com.br/blog/desenvolvimento-seguro-horusec>> Acesso em: 27 fev 2022.

ZUP. I.T Innovation. **Visão Geral Horusec.** 2022. Documentação Horusec. Disponível em: <<https://docs.horusec.io/pt-br/overview/>> Acesso em: 27 fev 2022.